

Base de datos del Agente de Eventos

Guía de conexión y uso · Proyecto Final Grupo 5 · UTEC — Diseño e Implementación de Agentes IA
Servicio administrado por Javier Díaz · Documento generado el 12 de setiembre de 2026

Qué es este servicio. Una base PostgreSQL 16 en Google Cloud que provee las tres capas de persistencia del agente: datos transaccionales (catálogo, cobertura, cotizaciones), recuperación semántica del corpus de conocimiento y memoria de largo plazo. Ustedes reciben credenciales de conexión; no necesitan cuenta en Google Cloud ni instalar herramientas de la nube. **El agente puede correr donde quieran** —laptop, Colab, Azure— porque la conexión es una URL estándar de PostgreSQL.

1. Qué recibes y qué hace cada archivo

Tu carpeta de entrega es personal e intransferible. Contiene cuatro elementos:

Archivo	Para qué sirve	¿Secreto?
<code>.env</code>	Tus credenciales: host, usuario, contraseña y la <code>DATABASE_URL</code> ya armada	SÍ
<code>certs/server-ca.pem</code>	Certificado del servidor: permite verificar que te conectas a la base correcta y no a un impostor	No (común a todos)
<code>certs/client-cert.pem</code>	Tu certificado de cliente: es el segundo factor de autenticación	SÍ
<code>certs/client-key.pem</code>	Tu llave privada. Nunca la compartas, ni siquiera con el equipo	SÍ
<code>datos.py</code>	La capa de acceso: las funciones que tu agente usará. No se modifica sin avisar al equipo	No

Por qué hay certificados además de contraseña. La base exige `TRUSTED_CLIENT_CERTIFICATE_REQUIRED` : sin tu certificado **no entras aunque tengas la contraseña correcta**. Funciona como un doble factor —algo que sabes y algo que tienes— y por eso podemos aceptar conexiones desde sus casas sin exponer la base. Está verificado: probamos conectar sin certificado y el servidor rechaza la conexión.

2. Puesta en marcha (4 pasos, una sola vez)

Paso 1 · Instalar dependencias

```
pip install "psycopg[binary,pool]" python-dotenv
```

Paso 2 · Colocar los archivos en tu proyecto

```
tu-proyecto/
├── .env           ← el de tu carpeta de entrega
├── datos.py       ← la capa de acceso
├── agente.py      ← tu código
├── certs/
│   ├── server-ca.pem
│   ├── client-cert.pem
│   └── client-key.pem
```

Paso 3 · Ajustar permisos (obligatorio)

```
chmod 600 certs/*.pem
```

Si los certificados tienen permisos abiertos, el cliente de PostgreSQL se niega a usarlos por seguridad. Es el error más frecuente al empezar.

Paso 4 · Verificar que todo funciona

```
python datos.py
```

Debe imprimir tu usuario, los tres esquemas con su número de tablas y los 6 productos del catálogo. Si lo ves, ya puedes trabajar.

Reglas no negociables. Agrega `.env` y `certs/` a tu `.gitignore` **antes del primer commit**. No pegues credenciales en el chat del grupo, en issues ni en notebooks compartidos. Si crees que tu llave se expuso, avisa de inmediato: se revoca solo la tuya y sigues trabajando en minutos.

3. Qué hay dentro: las tres capas

La base tiene **16 tablas** repartidas en tres esquemas, cada uno con un propósito distinto. Piénsalos como tres sistemas que comparten instancia.

core Verdad transaccional – 11 tablas

El negocio: qué se vende, dónde, cuándo y a qué precio. Es la única fuente válida de importes.

Tabla	Qué contiene
productos	Catálogo activo con SKU, servicio, capacidad y precio base
parametros	Las reglas del negocio con vigencia temporal: anticipación mínima, recargos, IGV, vigencia de cotización
cobertura	Qué distrito se atiende para qué servicio
disponibilidad	Cupos por producto y fecha
tipos_evento · producto_evento	Ocasiones y la afinidad de cada producto con cada una (insumo del motor de comparación)
clientes	Identificación mínima. El celular se guarda hasheado ; solo se conservan los 4 últimos dígitos
solicitudes	La necesidad del cliente: servicio, fecha, distrito, piso, ascensor, asistentes
cotizaciones · cotizacion_detalle	Cabecera con subtotal, IGV, total y vigencia; y el desglose línea por línea
derivaciones	Pases a un asesor humano, con el contexto estructurado obligatorio

rag Conocimiento estable – 2 tablas

documents	El documento padre del corpus, con versión y estado
chunks	Los fragmentos del corpus con su metadata (categoría, servicio, tipo de contenido) y su embedding vectorial para búsqueda semántica

memory Memoria de largo plazo – 3 tablas

consentimientos	Autorización explícita del cliente, separada entre obligatoria y promocional
preferencias_cliente	Lo aprendido sobre cada cliente: texto libre + su embedding + nivel de confianza. Requiere consentimiento registrado o la fila no puede existir
episodios	En qué etapa terminó cada conversación (para analizar abandono)

El estado de la conversación en curso lo maneja el **checkpoint** de LangGraph, que crea sus propias tablas la primera vez que se ejecuta.

4. Qué datos ya están cargados

Catálogo (6 productos activos)

Parámetros vigentes

SKU		Precio	Regla		Valor
BARRIL30 — Chopp 30 L + máquina		S/ 690.00	Anticipación mínima		72 h
BARRIL50 — Chopp 50 L + máquina		S/ 990.00	Piso máximo sin ascensor		3
EVENT0_BASICO — cerveza + mesas		S/ 2,500.00	Recargo piso 3 sin ascensor		S/ 100.00
EVENT0_PREMIUM — + equipos de frío		S/ 4,200.00	Recargo por unidad adicional		S/ 80.00
BOT_PERS_24 — botellas × 24		S/ 420.00	Personalización por botella		S/ 2.40
BOT_PERS_48 — botellas × 48		S/ 790.00	IGV		18 %
			Vigencia de cotización		48 h

Cobertura y disponibilidad

15 distritos de Lima: Ate, Barranco, Chorrillos, Jesús María, La Molina, Lince, Magdalena, Miraflores, Pueblo Libre, San Borja, San Isidro, San Juan de Lurigancho, San Miguel, Surco y Surquillo.

La cobertura **no es igual para los tres servicios**: barril llega a 12 distritos, eventos a 14 y botellas a 15. Eso es deliberado — permite probar los casos en que el agente debe decir que no.

Disponibilidad cargada para 61 días (hasta el 11 de noviembre de 2026), con menos cupos los fines de semana: 4 contra 10 entre semana.

Casos de prueba ya cargados

Identificador	Escenario	Lo que responde la base
demo-caso-2	Chopp 50 L, Surco, piso 2 sin ascensor, en 10 días	COTIZABLE
demo-caso-8	Chopp, Miraflores, piso 5 sin ascensor	Piso 5 sin ascensor supera el máximo operativo (3)

5. Cómo se usa en el agente

Regla de oro: el agente nunca escribe SQL. Todo pasa por `datos.py`, que expone una función por intención. Las tools de LangChain solo envuelven esas funciones.

Las funciones disponibles

Función	Qué hace
<code>consultar_catalogo(servicio)</code>	Productos activos. Única fuente válida de precios
<code>verificar_cobertura(distrito, servicio)</code>	Si se atiende esa zona. Llamar siempre antes de cotizar
<code>consultar_disponibilidad(sku, fecha)</code>	Si hay cupo. Si no hay registro, no asumir que sí
<code>registrar_solicitud(...)</code>	Guarda la necesidad y devuelve su id
<code>motivo_no_cotizable(solicitud_id)</code>	Devuelve el motivo del rechazo, o <code>None</code> si todo cumple
<code>emitir_cotizacion(solicitud_id, items)</code>	Calcula con precios y parámetros oficiales, y persiste
<code>derivar(cliente_id, motivo, contexto)</code>	Pase a un asesor con el contexto obligatorio
<code>buscar_conocimiento(embedding, servicio)</code>	Recupera del corpus filtrando por servicio antes de la similitud
<code>recordar(cliente_id, embedding)</code>	Preferencias vigentes del cliente
<code>guardar_preferencia(...)</code>	Persiste solo si hay consentimiento y no es duplicada
<code>registrar_episodio(...)</code>	Deja constancia de dónde terminó la conversación

Ejemplo: envolver una función en una tool

```
from langchain.tools import tool
import datos

@tool
def consultar_catalogo(servicio: str = "") -> str:
    """Productos activos del catálogo oficial. Única fuente válida de
    precios: nunca menciones productos ni importes que no aparezcan aquí."""
    return str(datos.consultar_catalogo(servicio or None))

@tool
def cotizar(solicitud_id: int, sku: str, cantidad: int) -> str:
    """Emite la cotización oficial. Si devuelve NO COTIZABLE, explica el
    motivo al cliente y deriva: no intentes calcular un importe por tu cuenta."""
    motivo = datos.motivo_no_cotizable(solicitud_id)
    if motivo:
        return f"NO COTIZABLE: {motivo}. Derivar a un asesor."
    return str(datos.emitir_cotizacion(solicitud_id, [(sku, cantidad)]))
```

El docstring de la tool es el prompt que ve el modelo. Es lo único con lo que decide cuándo usarla. Escríbelo con cuidado: ahí se le dice, por ejemplo, que no invente precios.

Memoria de conversación (no repreguntar)

```
from langgraph.checkpoint.postgres import PostgresSaver
import os

with PostgresSaver.from_conn_string(os.environ["DATABASE_URL"]) as saver:
    saver.setup() # crea sus tablas la primera vez
    grafo = builder.compile(checkpointer=saver)
```

Esto es lo que permite que el agente recuerde lo que el cliente ya dijo y sobreviva a un cambio de intención a mitad de conversación.

6. Las reglas que la base aplica sola

Esto es importante para entender el comportamiento del agente: **varias reglas de negocio no están en el prompt, están en la base de datos**, y por eso no se pueden negociar en una conversación.

Regla	Qué pasa si se intenta violar
Piso 4 o superior sin ascensor	La base rechaza la cotización con el motivo exacto
Menos de 72 h de anticipación	Rechazada; hay que derivar
Distrito fuera de cobertura	Rechazada
Total distinto de subtotal + impuestos	Rechazada: la aritmética se valida
Guardar una preferencia sin consentimiento	Imposible: la tabla lo exige estructuralmente
Derivar sin contexto completo	Rechazada si faltan los campos obligatorios

Cuando una de estas reglas se activa, `datos.py` devuelve el motivo en texto claro. **El agente debe explicárselo al cliente y derivar**, nunca intentar rodearla ni improvisar un importe.

7. Qué puedes hacer y qué no

Capa	Tu usuario personal	El agente en ejecución
<code>core</code> (catálogo, cotizaciones)	Leer y escribir	Solo leer; escribe únicamente solicitudes, cotizaciones y derivaciones
<code>rag</code> (corpus)	Leer y escribir (para ingestar chunks)	Solo leer
<code>memory</code> (preferencias)	Solo leer	Leer y escribir

Ningún usuario puede borrar el catálogo ni eliminar tablas. Está verificado: el usuario del agente intentó un `DELETE` sobre productos y fue denegado.

8. Problemas frecuentes

Mensaje	Causa y solución
<code>connection requires a valid client certificate</code>	Falta el certificado o la ruta es incorrecta. Revisa que <code>certs/</code> esté donde dice tu <code>.env</code>
<code>private key file has group or world access</code>	Ejecuta <code>chmod 600 certs/*.pem</code>
<code>could not connect to server /timeout</code>	La instancia probablemente está apagada para ahorrar costo. Consulta al grupo antes de asumir una falla
<code>permission denied for table ...</code>	Estás intentando algo fuera de tu alcance (ver sección 7). Si lo necesitas, pídelo
No se puede cotizar automáticamente: ...	No es un error: es una regla de negocio funcionando. Explícala y deriva
<code>missing "=" after ...</code> al conectar	La ruta de tus certificados tiene espacios. Usa parámetros separados en vez de una cadena única

9. Dónde puede correr tu agente

Entorno	Qué hacer con los certificados
Tu laptop	La carpeta <code>certs/</code> junto al código. Nada más
Google Colab	Sube los <code>.pem</code> a <code>/content/certs/</code> y guarda la <code>DATABASE_URL</code> en los Secrets del notebook
Azure / Cloud Run / contenedor	Los <code>.pem</code> como secretos montados; la <code>DATABASE_URL</code> como variable de entorno
GitHub Actions	Certificados en secrets del repositorio, escritos a disco en un paso previo

`datos.py` no importa ninguna librería de Google: solo depende de `DATABASE_URL`. El mismo código funciona en los cuatro entornos.